

(Disclaimer: this document was outlined by an LLM model)

Objective: Design and implement a locally hosted AI system that develops a personality and maintains long-term contextual awareness without exceeding token limits.

Methodology: I transitioned from a basic local LLM instance to an automated, dual-model "Actor-Observer" architecture using Python and Ollama. This setup allows the system to reflect on user interactions, shift its own tonal parameters, and manage a bi-level memory system mimicking human cognition.

Project Timeline & Implementation Phases

Environment Provisioning

Phase 1

I set up the foundational local AI environment by installing Ollama. This allowed me to pull and run base models (like Llama 3) directly through my local command line, verifying that the core generation capabilities were functioning offline.

Actor-Observer Architecture

Phase 2

I replaced manual terminal interactions with a Python orchestrator script. I architected a dual-model system:

- **The Actor:** The primary AI that I interact with directly.
- **The Observer:** A secondary background model that silently reads the chat transcript, extracts one-sentence memories, and generates personality updates in strict JSON format. The script then dynamically injects these updates into the Actor's system prompt before the next turn, allowing the personality to evolve without manual intervention.

Memory Compression Thresholds

Phase 3

To solve the inevitable context window bloat, I introduced an automated compression trigger. Once the system accumulated a set number of raw memory logs (e.g., 10 entries), the orchestrator paused the chat loop and routed the logs back to the Observer. The Observer synthesized these data points into a single, dense "core memory" paragraph, wiping the bloated logs and freeing up tokens.

Temporal Memory Simulation

Phase 4

I finalized the architecture by moving from volume-based compression to time-based consolidation, mimicking human short-term and long-term memory.

- I implemented a local JSON store to tag all new memories with exact timestamps.
- **Short-Term Buffer:** Memories less than 48 hours old remain granular, giving the AI high-fidelity recall for rapid, successive queries within a single day or coding session.
- **Long-Term Consolidation:** Once memories age past the 48-hour threshold, the script automatically feeds them to the Observer alongside the existing long-term history to be synthesized into a permanent, highly abstracted biography.

Core Findings & Next Steps

By decoupling the interaction layer (Actor) from the cognitive maintenance layer (Observer), I effectively removed the cognitive load from the primary model. The system successfully curates a rolling, token-efficient history while allowing the AI's demeanor to genuinely reflect our ongoing interactions.